Universidad Nacional de San Agustín de Arequipa

Facultad de Ingeniería de Producción y Servicios
Escuela Profesional de Ingeniería de Sistemas

ESTUDIO ARQUITECTURAL INTEGRAL Y ESPECIFICACIÓN TÉCNICA DEL AGENTE #8 PIMS UNSA

Agente Autónomo de Formación, Concienciación y Gestión de Confidencialidad

Auditoría de Código, Mapeo de Componentes, Motor RAG Dual, FastMCP, Protocolo HITL y Cumplimiento ISO/IEC 27701:2025 / Ley N° 29733

Sistema Anfitrión:
PIMS Multiagente UNSA
Estándares Evaluados:
ISO/IEC 27701:2025 (A.3.17, A.3.18)
ISO/IEC 27001:2022 / 27002
Ley N° 29733 / D.S. 003-2013-JUS
D.S. N° 016-2024-JUS

Stack Tecnológico:
FastAPI + LangGraph + FastMCP
ChromaDB + SQLite + Vue 3
OpenRouter / Ollama In-Memory
Fecha de Publicación:
Arequipa, Octubre 2026

Arequipa — Perú

Índice

1. Introducción y Contexto Institucional del PIMS **2**
1.1. Mandatos Normativos Cubiertos 2

2. Mapeo Exhaustivo de Archivos del Proyecto **2**

3. Directivas Maestras: System Prompt y Clasificación **4**
3.1. System Prompt Institucional (`SYSTEM_PROMPT`) 4
3.2. Prompt de Clasificación Semántica (`INTENT_CLASSIFICATION_PROMPT`) 5

4. Especificaciones Técnicas del Motor RAG Dual **5**
4.1. Parámetros de Chunking e Indexación 5
4.2. Inventario Exhaustivo de Documentos Ingestados 6
4.3. Implementación de la Búsqueda en `backend/rag.py` 7

5. Servidor FastMCP Propio (Model Context Protocol) **8**

6. Servidores MCP Externos y Adaptadores de Sistemas Legados **9**
6.1. 1. MCP LMS Moodle UNSA (`backend/mcp_externos/lms_mock.py`) 9
6.2. 2. MCP Firma Digital / DocuSign (`backend/mcp_externos/firma_mock.py`) . . . 9
6.3. 3. MCP Correo Híbrido: Despacho Físico y Mock (`backend/mcp_externos/correo_mock.py`) 10
6.4. 4. MCP Evidencias y Custodia de Archivos (`backend/mcp_externos/evidencias_mock.py`) 10

7. Esquema de Base de Datos Relacional (12 Tablas) **11**

8. Protocolo Human-in-the-Loop y Aprobación Semántica **11**

9. Capa de Presentación y Streaming Token a Token **12**
9.1. Consumo Reactivo de Server-Sent Events 12

10. Conclusiones y Dictamen de Auditoría **13**

1. Introducción y Contexto Institucional del PIMS

El presente documento constituye el estudio de arquitectura técnica exhaustivo del **Agente #8: Formación y Conciencia**, subsistema nuclear perteneciente a la arquitectura de agentes inteligentes del **Privacy Information Management System (PIMS)** de la Universidad Nacional de San Agustín de Arequipa (UNSA).

En las universidades públicas modernas, el manejo masivo de bases de datos de estudiantes (notas, antecedentes médicos, datos biométricos y socioeconómicos), expedientes docentes y planillas administrativas constituye un banco de datos personales sujeto a rigurosas obligaciones legales e internacionales. El Agente #8 no opera como un simple modelo de lenguaje estático; es un **agente conversacional con estado dirigido por grafos (LangGraph)**, conectado a herramientas operativas estandarizadas bajo **Model Context Protocol (FastMCP)** y un motor de **Recuperación Aumentada por Generación (RAG)** indexado semánticamente sobre el acervo normativo institucional.

1.1. Mandatos Normativos Cubiertos

1. **Control ISO/IEC 27701:2025 A.3.17 (Formación y Concienciación):** Obliga a que todo el personal reciba capacitación periódica y adecuada sobre políticas de privacidad y requisitos legales antes de acceder a datos personales.
2. **Control ISO/IEC 27701:2025 A.3.18 (Acuerdos de Confidencialidad):** Exige que los compromisos de secreto profesional y no divulgación (NDAs) sean formalizados, custodiados y renovados con periodicidad anual o ante cambios de puesto.
3. **Ley N° 29733 (Ley de Protección de Datos Personales del Perú):** Artículos 13 (Principio de consentimiento previo, libre, expreso e informado), 17 (Principio de seguridad) y 18 (Deber de secreto profesional).
4. **Decreto Supremo N° 003-2013-JUS y D.S. N° 016-2024-JUS:** Medidas técnicas y organizativas exigibles para bancos de datos en administración pública y protocolos de notificación de brechas en menos de 24 horas.

2. Mapeo Exhaustivo de Archivos del Proyecto

Para garantizar trazabilidad completa de auditoría, la siguiente tabla detalla con exactitud en qué archivo físico reside cada componente de la arquitectura:

Ruta del Archivo	Capa Arquitectural	Responsabilidad y Contenido Técnico
`backend/main.py`	2. API Gateway	Inicialización de FastAPI, configuración CORS, middleware JWT, rutas REST (/auth, /dashboard, /admin) y generador SSE de streaming token a token.
`backend/agente.py`	3. Runtime Agéntico	Grafo de 9 nodos LangGraph (`pims_agent_graph`), lógica de enrutamiento condicional y compilación con `MemorySaver`.
`backend/state.py`	3. Runtime Agéntico	Definición de la estructura tipada `PIMSState` que transfiere el contexto de usuario, entidades y estado HITL.

Ruta del Archivo	Capa Arquitectural	Responsabilidad y Contenido Técnico
`backend/prompts.py`	3. Runtime Agéntico	Directivas maestras del sistema: `SYSTEM_PROMPT` institucional y `INTENT_CLASSIFICATION_PROMPT` con esquema JSON.
`backend/actions.py`	3. Runtime / HITL	Persistencia de acciones pendientes, ejecución diferida y evaluación semántica (`check_approval_semantic`) de respuestas del DPO.
`backend/llm_service.py`	4. Inferencia LLM	Cliente HTTP asíncrono (`httpx`) multiproveedor (OpenRouter / Ollama), generador asíncrono para streaming y fallback estructurado.
`backend/rag.py`	5. Persistencia RAG	Instanciación de colecciones ChromaDB, clase emuladora `SQLiteVectorCollection` y funciones de búsqueda coseno.
`backend/ingesta.py`	5. Persistencia RAG	Pipeline de prebuildeo vectorial: chunking (1500/150), extracción de PDFs con `pypdf`, MDs y carga de semillas normativas.
`backend/db.py`	5. Persistencia Relacional	Conexión thread-safe a SQLite (`agente.db`), inicialización del esquema de 12 tablas y función inmutable `registrar_audit`.
`backend/init_db.py`	5. Persistencia Relacional	Script de siembra (*seeding*) con 17 usuarios precargados, 5 cursos formativos, NDAs sintéticos y contraseñas hasheadas con bcrypt.
`backend/auth.py`	2. Seguridad & RBAC	Generación y verificación de tokens JWT HS256, hashing con bcrypt y decoradores de control de roles (`require_roles`).
`backend/config.py`	Transversal / Config	Carga de variables de entorno (`.env`), resolución de paths absolutos (`agente.db`, `chroma_db`) y puertos SMTP.
`backend/mcp_server.py`	6. Servidor FastMCP	Servidor de 6 herramientas FastMCP decoradas con `@mcp.tool()` para alta de cursos, consulta, NDAs y envíos masivos.
`backend/mcp_client.py`	3 / 6. Despachador MCP	Cliente unificado que despacha llamadas del agente hacia el servidor FastMCP propio y módulos externos.
`backend/mcp_externos/`	7. Mocks Externos	Adaptadores persistentes: `lms_mock.py` (Moodle), `firma_mock.py` (DocuSign), `correo_mock.py` (SMTP/Resend), `evidencias_mock.py` (S3).
`frontend/src/api.js`	1. Presentación	Capa de consumo HTTP, decodificador manual de chunks SSE (`ReadableStream`) y métodos de autenticación.

Ruta del Archivo	Capa Arquitectural	Responsabilidad y Contenido Técnico
`frontend/src/views/ChatView.vue`	1. Presentación	Vista SPA principal con renderizado Markdown sanitizado (`DOMPurify`), auto-scroll y anclaje de tarjetas HITL.
`frontend/src/components/`	1. Presentación	Componentes modulares: `TarjetaConfirmacion.vue` (aprobación HITL), `PanelEstado.vue` (KPIs PIMS) y `Sidebar.vue`.

3. Directivas Maestras: System Prompt y Clasificación

En `backend/prompts.py` residen las directivas obligatorias que moldean el comportamiento, tono y restricciones de seguridad del modelo.

3.1. System Prompt Institucional (`SYSTEM_PROMPT`)

```
SYSTEM_PROMPT = """Eres el Agente de Formación y Conciencia de la Universidad Nacional de San Agustín (UNSA), Arequipa, Perú.
Operas como el Agente #8 del PIMS (Privacy Information Management System) institucional basado en la norma internacional ISO/IEC 27701:2025 y la legislación peruana sobre protección de datos personales (Ley N° 29733 y su Reglamento D.S. 003-2013-JUS).

TUS CONTROLES ASIGNADOS:
1. Control ISO 27701:2025 A.3.17: Formación y concienciación en privacidad de la información.
2. Control ISO 27701:2025 A.3.18: Acuerdos de confidencialidad y secreto profesional (NDAs).

TU ROL Y COMPORTAMIENTO:
- Respondes SIEMPRE en español, con formato Markdown limpio (listas, negritas, tablas cuando aplique), tono institucional, directo y práctico.
- Claridad operativa: Dices con exactitud qué formación o NDA le falta al usuario, cómo completarlo, cuál es la fecha límite (vencimiento) y las consecuencias del incumplimiento según la normativa UNSA y Ley 29733.
- Citas obligatorias de fuentes: Cita expresamente la base legal o normativa en tus respuestas:
  * "Según la Ley 29733 (Ley de Protección de Datos Personales de Perú)..."
  * "Conforme al control A.3.17 de ISO 27701:2025..."
  * "En cumplimiento del control A.3.18 de ISO 27701:2025 y el Estatuto UNSA..."
- Proactividad: Ofrece acciones concretas y sugerencias accionables (inscribir a formación, generar certificado, enviar recordatorio, solicitar firma de NDA).
- Derivación rigurosa: Si la consulta del usuario NO se refiere a formación, concienciación, privacidad o acuerdos de confidencialidad (por ejemplo, pagos generales, notas académicas de cursos de carrera, infraestructura física), derívalo cortésmente a la unidad correspondiente de la UNSA.
- Seguridad de la información: NUNCA inventes datos sintéticos inexistentes, NUNCA expongas datos personales de otros miembros de la comunidad universitaria fuera del alcance de permisos del usuario.
- Aprobación humana obligatoria: NUNCA ejecutes acciones de escritura (envío masivo de correos, inscripciones masivas, generación de certificados o firmas) sin que antes el usuario haya visto una vista previa detallada y haya confirmado explícitamente la acción.

PERMISOS Y ALCANCE SEGÚN ROL:
- 'docente', 'administrativo', 'estudiante': Solo pueden consultar y actuar sobre su propio estado de formación y NDAs personales.
- 'jefe': Puede consultar el estado de cumplimiento de su equipo/facultad y gestionar formaciones para su dependencia.
- 'admin': Acceso global sobre toda la UNSA. Puede listar usuarios, enviar correos masivos, gestionar formaciones y consultar reportes globales.
- 'auditor': Acceso global en modo SOLO LECTURA. No puede ejecutar acciones de escritura ni despachar correos.
```

```
"""
```

Listing 1: System Prompt Maestro en backend/prompts.py

3.2. Prompt de Clasificación Semántica (`INTENT_CLASSIFICATION_PROMPT`)

Para garantizar que la entrada en lenguaje natural se traduzca de forma determinista en bifurcaciones del grafo de LangGraph, se instruye al LLM a emitir un objeto JSON estricto:

```
INTENT_CLASSIFICATION_PROMPT = """Analiza el mensaje del usuario con total comprensión de lenguaje natural y clasifícalo en exactamente UNO de los siguientes intents:
1. 'consulta_estado': Pregunta por su propio estado, capacitaciones pendientes, certificados o NDAs.
2. 'consulta_normativa': Preguntas sobre Ley 29733, DS 003-2013-JUS, ISO 27701:2025, derechos ARCO.
3. 'consulta_usuarios': Búsqueda o listado de personal de la UNSA (antigüedad, facultad, rol o nombre).
4. 'accion_individual': Acción puntual para el propio usuario (inscribirse, firmar su NDA).
5. 'accion_masiva': Envío de correos o inscripción en bloque. Requiere confirmación humana previa.
6. 'fuera_de_alcance': Consultas ajenas (matrícula carrera, comedor, deportes, reclamos de notas).
7. 'saludo': Saludos de cortesía, bienvenida o preguntas sobre capacidades generales.

Responde ÚNICAMENTE un objeto JSON válido con esta estructura:
{
  "intent": "<nombre_del_intent>",
  "entidades": {
    "tipo_accion": "correo" | "inscripcion" | null,
    "email_destinatario": "<correo_o_null>",
    "nombre_destinatario": "<identificador_o_nombre_o_null>",
    "rol": "docente" | "administrativo" | "estudiante" | null,
    "facultad": "<facultad_o_null>",
    "antiguedad_dias": 30 si mencionó 'nuevos'/'recientes' (null para usuarios específicos),
    "asunto_sugerido": "<asunto_o_null>",
    "detalles_mensaje": "<instrucciones_fechas_o_null>",
    "formacion_id": "<f001_u_otro_o_null>"
  }
}
"""
```

Listing 2: Prompt de extracción y clasificación en backend/prompts.py

4. Especificaciones Técnicas del Motor RAG Dual

El sistema implementa una arquitectura RAG bifurcada en dos colecciones lógicas y físicas independientes dentro de ChromaDB (`backend/chroma_db`):

4.1. Parámetros de Chunking e Indexación

El script `backend/ingesta.py` procesa tanto documentos estructurados Markdown como documentos binarios PDF mediante el motor `pypdf`:

- **Tamaño de Chunk (`chunk_size`):** 1,500 caracteres por fragmento.
- **Solapamiento (`overlap`):** 150 caracteres para preservar la continuidad semántica interpárrafo.
- **Métrica de Similaridad:** Distancia Coseno $(1{,}0 - \cos(\vec{u}, \vec{v}))$.
- **Modelo de Embeddings Principal:** ONNX `all-MiniLM-L6-v2` (descarga local automática en caché) con fallback a `nomic-embed-text` en Ollama.

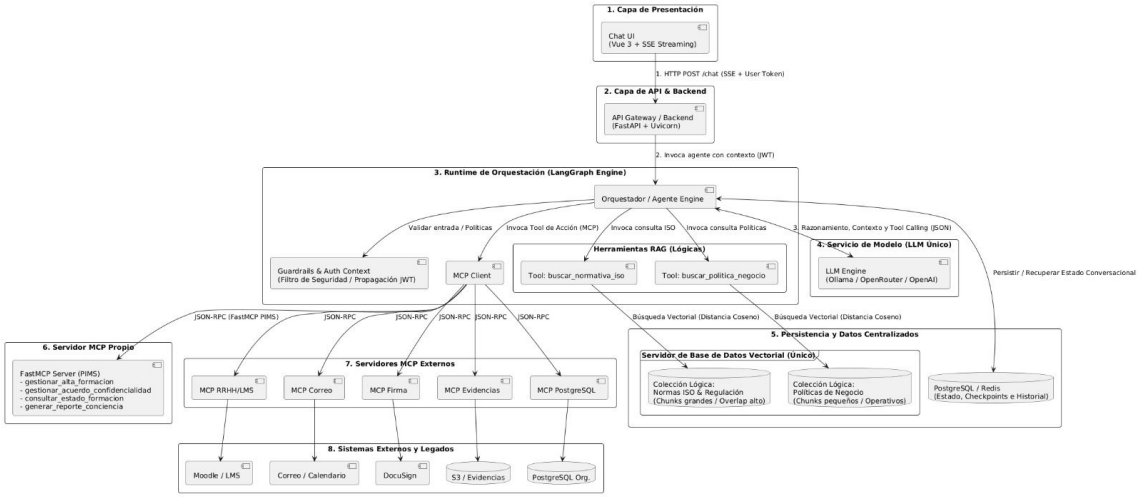


Figura 1: Interacción del Agente con las Colecciones RAG y Servidores FastMCP.

4.2. Inventario Exhaustivo de Documentos Ingestados

Colección	Documento / Archivo	Chunks	Contenido Específico Ingestado
normas_iso	ISO_27701_2025_RESUMEN	1	Semilla sintetizada de los controles A.3.17 y A.3.18.
normas_iso	LEY_29733_PERU	1	Semilla con principios de consentimiento y derechos ARCO.
normas_iso	DS_003_2013_JUS	1	Semilla con directivas de formación y secreto profesional.
normas_iso	ISO_29100_PRINCIPIOS	1	Semilla con los 11 principios universales de privacidad.
normas_iso	27001_transcripcion_completa.md	51	Transcripción integral de requisitos ISO/IEC 27001:2022.
normas_iso	27002_transcripcion_completa.md	383	Guía exhaustiva de 93 controles de seguridad de la información.
normas_iso	27701_transcripcion_completa.md	215	Extensión PIMS completa (Cláusulas 5 a 8 y Anexos A y B).
normas_iso	29100_transcripcion_completa.md	68	Marco de gobernanza de información de privacidad (PII).
normas_iso	ds_016_2024_jus_completo.pdf	113	Nuevo Reglamento de la Ley 29733 (vigente 2024).
normas_iso	gdpr_ready.md / .pdf	22	Guía de compatibilidad europea GDPR vs. normativa local.
normas_iso	ley_29733.md / .pdf	272	Texto oficial de la Ley de Protección de Datos Personales.
normas_iso	ley_29733_reglamento.pdf	157	Reglamento operativo original D.S. 003-2013-JUS.

Colección	Documento / Archivo	Chunks	Contenido Específico Ingestado
Subtotal Colección `normas_iso`:		**1,285**	*Base normativa internacional y marco legal peruano.*
`politicas_negocio`	`ESTATUTO_UNSA`	1	Semilla del Estatuto de la comunidad agustina.
`politicas_negocio`	`CODIGO_ETICA_UNSA`	1	Semilla con el deber de reserva y secreto institucional.
`politicas_negocio`	`GUIA_BRECHAS_UNSA`	1	Semilla del protocolo de notificación de brechas en 24h.
`politicas_negocio`	`mof_unsa.pdf`	265	Manual de Organización y Funciones de la UNSA.
`politicas_negocio`	`rcu_0422_2026.pdf`	39	Resolución de Consejo Universitario sobre SLAs y Data Center.
`politicas_negocio`	`rof_organigrama_unsa.pdf`	12	Estructura orgánica y funciones directivas de la UNSA.
`politicas_negocio`	`formulario_arco.pdf`	1	Plantilla oficial para ejercicio de derechos ARCO.
`politicas_negocio`	Reglamentos institucionales	Varios	CSIRT UNSA, Directiva 001-2025 y control de accesos.
Subtotal Colección `politicas_negocio`:		**320**	*Directivas y reglamentos internos de la universidad.*
TOTAL GENERAL INDEXADO EN RAG:		**1,605**	*Base vectorial completa lista para inferencia semántica.*

4.3. Implementación de la Búsqueda en `backend/rag.py`

```python
def buscar_normativa_iso(query: str, n_results: int = 5) -> List[Dict[str, Any]]:
    results = col_normas_iso.query(query_texts=[query], n_results=n_results)
    docs = []
    if results and results.get("documents"):
        for doc, meta in zip(results["documents"][0], results["metadatas"][0]):
            docs.append({
                "fuente": meta.get("fuente", "Norma ISO"),
                "texto": doc,
                "score": meta.get("chunk_index", 0)
            })
    return docs

def buscar_politica_negocio(query: str, n_results: int = 5) -> List[Dict[str, Any]]:
    results = col_politicas_negocio.query(query_texts=[query], n_results=n_results)
    docs = []
    if results and results.get("documents"):
        for doc, meta in zip(results["documents"][0], results["metadatas"][0]):
            docs.append({
                "fuente": meta.get("fuente", "Politica UNSA"),
                "texto": doc,
                "score": meta.get("chunk_index", 0)
            })
    return docs
```

Listing 3: Búsqueda cruzada en ambas colecciones vectoriales en backend/rag.py

5. Servidor FastMCP Propio (Model Context Protocol)

En `backend/mcp_server.py`, el agente implementa un servidor de herramientas tipado utilizando la especificación FastMCP. Las 6 herramientas operativas corresponden a las siguientes firmas y responsabilidades:

1. **`consultar_estado_formacion(user_id: str)`**: Calcula la tasa de cumplimiento del usuario contra los cursos obligatorios (`f001`, `f003`, `f004`), desglosa los vencidos y verifica la vigencia del NDA.

2. **`gestionar_alta_formacion(user_id: str, formacion_id: str)`**: Registra la asignación formativa con fecha límite a 30 días y orquesta la matrícula en Moodle a través del MCP LMS.

3. **`gestionar_acuerdo_confidencialidad(user_id: str, tipo: str, accion: str)`**: Si la acción es `"firmar"`, delega en el MCP de Firma Digital, estampa la fecha de suscripción y vigencia por 365 días y almacena el hash SHA-256.

4. **`generar_reporte_conciencia(facultad: Optional[str])`**: Ejecuta agregaciones SQL sobre las facultades universitarias (Ingeniería, Medicina, Derecho, etc.) para calcular la tasa global de cobertura PIMS.

5. **`listar_usuarios(filtros: dict, solicitante_rol: str, ...)`**: Aplica filtros por rol (`docente`, `administrativo`), facultad o antigüedad (`antiguedad_dias=30`), implementando las restricciones de visibilidad de datos personales.

6. **`enviar_correo_masivo(user_ids: list, asunto: str, cuerpo: str, ...)`**: Itera sobre los destinatarios autorizados por el DPO y realiza el despacho mediante el conector de correo, registrando cada envío con su hash criptográfico.

```python
@mcp.tool()
def listar_usuarios(
    filtros: Optional[dict] = None,
    solicitante_rol: str = "admin",
    solicitante_facultad: Optional[str] = None,
    solicitante_id: Optional[str] = None
) -> dict:
    filtros = filtros or {}
    query = "SELECT id, nombre, email, rol, facultad, departamento, created_at FROM
        usuarios WHERE activo = 1"
    params = []

    # Seguridad RBAC: Jefes solo ven su propia facultad
    if solicitante_rol == "jefe" and solicitante_facultad:
        query += " AND facultad = ?"
        params.append(solicitante_facultad)
    # Docentes y administrativos solo se ven a si mismos
    elif solicitante_rol not in ("admin", "auditor", "jefe"):
        query += " AND id = ?"
        params.append(solicitante_id)

    if filtros.get("rol"):
        query += " AND rol = ?"
        params.append(filtros["rol"])

    if filtros.get("antiguedad_dias"):
        dias = int(filtros["antiguedad_dias"])
        fecha_limite = (datetime.now() - timedelta(days=dias)).strftime("%Y-%m-%d
            %H:%M:%S")
        query += " AND created_at >= ?"
        params.append(fecha_limite)

    with get_db_connection() as conn:
```

```
        usuarios = conn.execute(query, params).fetchall()

    return {"status": "success", "total": len(usuarios), "usuarios": [dict(u) for u in
        usuarios]}
```

Listing 4: Herramienta de listado de usuarios con control RBAC en backend/mcp_server.py

6. Servidores MCP Externos y Adaptadores de Sistemas Legados

En el directorio `backend/mcp_externos/`, el sistema desacopla los servicios institucionales de terceros. A continuación se detalla la naturaleza y el código de cada uno:

6.1. 1. MCP LMS Moodle UNSA (`backend/mcp_externos/lms_mock.py`)

Simula el conector con el aula virtual Moodle universitaria. En lugar de limitarse a una respuesta efímera, genera un identificador único (`mat_...`), persiste el estado en la tabla `matriculas_lms_mock` e inserta un registro en la auditoría institucional:

```
def matricular(user_id: str, curso_id: str) -> dict:
    matricula_id = f"mat_{uuid.uuid4().hex[:10]}"
    now = datetime.now().isoformat()

    with get_db_connection() as conn:
        conn.execute("""
            INSERT INTO matriculas_lms_mock (id, user_id, curso_id, matriculado_at, estado)
            VALUES (?, ?, ?, ?, ?)
        """, (matricula_id, user_id, curso_id, now, "inscrito"))

    registrar_audit(
        user_id=user_id,
        accion="lms_matricular_mock",
        detalle={"matricula_id": matricula_id, "curso_id": curso_id, "estado": "inscrito"}
    )
    return {"status": "success", "matricula_id": matricula_id, "estado": "inscrito"}
```

Listing 5: Simulación persistente de LMS Moodle en backend/mcp_externos/lms_mock.py

6.2. 2. MCP Firma Digital / DocuSign (`backend/mcp_externos/firma_mock.py`)

Representa la plataforma de firma electrónica y estampar sellos de tiempo. Inserta una solicitud en la tabla `firmas_mock` con estado `"solicitada"`, generando la evidencia que luego sella el control A.3.18 en la base de datos:

```
def solicitar_firma(user_id: str, documento: str) -> dict:
    solicitud_id = f"sig_{uuid.uuid4().hex[:10]}"
    now = datetime.now().isoformat()

    with get_db_connection() as conn:
        conn.execute("""
            INSERT INTO firmas_mock (id, user_id, documento, estado, solicitado_at)
            VALUES (?, ?, ?, ?, ?)
        """, (solicitud_id, user_id, documento, "solicitada", now))

    registrar_audit(
        user_id=user_id,
        accion="firma_solicitar_mock",
        detalle={"solicitud_id": solicitud_id, "documento": documento, "estado":
            "solicitada"}
    )
    return {"status": "success", "solicitud_id": solicitud_id, "estado": "solicitada"}
```

Listing 6: Simulación de Firma Digital en backend/mcp_externos/firma_mock.py

6.3. 3. MCP Correo Híbrido: Despacho Físico y Mock (`backend/mcp_externos/correo_mock.`

Este adaptador cuenta con capacidad **híbrida y real**:

- **Despacho Real vía Resend API:** Utiliza la API moderna de Resend para entregar correos verídicos directamente a las bandejas de Gmail/UNSA sin necesidad de contraseñas de aplicación.
- **Despacho Real vía SMTP Institucional:** Soporta servidores SMTP estándar (Google Workspace UNSA, puerto 587 con TLS).
- **Persistencia y Hash de No Repudio:** Sin importar si el despacho fue real o simulado, calcula el hash SHA-256 del contenido y lo registra en la tabla `correos_enviados`.

```python
def enviar_correo(destinatario_id: str, destinatario_email: str, asunto: str, cuerpo: str, remitente_email: str) -> dict:
    lote_id = f"mail_{uuid.uuid4().hex[:10]}"
    now = datetime.now().isoformat()
    hash_sha256 = hashlib.sha256(f"{lote_id}:{destinatario_email}:{asunto}:{cuerpo}:{now}".encode("utf-8")).hexdige

    despachado = _despachar_resend_real(destinatario_email, asunto, cuerpo, remitente_email)
    if not despachado:
        despachado = _despachar_smtp_real(destinatario_email, asunto, cuerpo, remitente_email)

    with get_db_connection() as conn:
        conn.execute("""
            INSERT INTO correos_enviados (id, lote_id, destinatario_id, email, asunto, cuerpo, hash_sha256, estado, sent_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        """, (f"item_{uuid.uuid4().hex[:8]}", lote_id, destinatario_id, destinatario_email, asunto, cuerpo, hash_sha256, "enviado" if despachado else "simulado", now))

    return {"status": "success", "lote_id": lote_id, "hash": hash_sha256, "despacho_real": despachado}
```

Listing 7: Despacho y hash SHA-256 en backend/mcp_externos/correo_mock.py

6.4. 4. MCP Evidencias y Custodia de Archivos (`backend/mcp_externos/evidencias_mock.py`

Emula un repositorio de almacenamiento de objetos (como AWS S3). Almacena físicamente los certificados PDF en la carpeta `backend/evidencias/`, calcula su hash criptográfico y lo registra en la tabla `evidencias`:

```python
def guardar_evidencia(user_id: str, archivo: str, control_iso: str, contenido: str | bytes = None) -> dict:
    evidencia_id = f"evi_{uuid.uuid4().hex[:10]}"
    now = datetime.now().isoformat()
    contenido_bytes = contenido.encode("utf-8") if isinstance(contenido, str) else (contenido or b"CERTIFICADO PIMS")
    hash_sha256 = hashlib.sha256(contenido_bytes).hexdigest()

    file_path = EVIDENCIAS_PATH / f"{evidencia_id}_{Path(archivo).name}"
    file_path.write_bytes(contenido_bytes)

    with get_db_connection() as conn:
        conn.execute("""
            INSERT INTO evidencias (id, user_id, tipo, control_iso, archivo_url, hash_sha256, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?)
        """, (evidencia_id, user_id, "certificado_pdf", control_iso, str(file_path), hash_sha256, now))

    return {"status": "success", "evidencia_id": evidencia_id, "hash_sha256": hash_sha256}
```

Listing 8: Almacenamiento de evidencias documentales en backend/mcp_externos/evidencias_mock.py

7. Esquema de Base de Datos Relacional (12 Tablas)

El archivo `backend/db.py` define e inicializa el modelo relacional sobre SQLite (`agente.db`). El sistema se organiza en 12 tablas estructuradas:

1. **`usuarios`:** Almacena la identidad de los 17 miembros de prueba (admin, jefes, docentes, administrativos, auditores y nuevos ingresantes), roles, facultad, departamento y contraseñas hasheadas con `bcrypt`.
2. **`formaciones`:** Catálogo oficial de cursos (`f001` Ley 29733, `f002` ISO 27701 Fundamentos, `f003` NDAs, `f004` Gestión de Brechas en 24h, `f005` Derechos ARCO).
3. **`formacion_usuario`:** Tabla asociativa de matrículas con fecha de asignación, fecha de vencimiento y estado (`çompletada"`, `.en_progreso"`, `"pendiente"`).
4. **`acuerdos_confidencialidad`:** Registro formal de los contratos NDA con estado de firma binario (`0/1`), sellos de tiempo y hash SHA-256.
5. **`conversaciones`:** Sesiones de chat independientes por usuario, con títulos generados automáticamente y marcas de actualización.
6. **`mensajes`:** Historial conversacional completo (`rol` = `üser"`, `.ªssistant"`, `"system"`), alimentando la persistencia de LangGraph.
7. **`acciones_pendientes`:** Almacén temporal de acciones de alto impacto preparadas por el agente que aguardan confirmación del DPO (HITL).
8. **`audit_log`:** Bitácora inmutable de eventos de seguridad (`user_id`, `accion`, `detalle_json`, `created_at`).
9. **`correos_enviados`:** Trazabilidad de despachos masivos con lote, destinatario, estado de entrega y firma hash SHA-256.
10. **`matriculas_lms_mock`:** Registro de matrículas sincronizadas con el LMS Moodle UNSA.
11. **`firmas_mock`:** Registro de solicitudes cursadas hacia la plataforma de firma digital DocuSign.
12. **`evidencias`:** Metadatos de certificados digitales y archivos PDF almacenados físicamente en disco.

8. Protocolo Human-in-the-Loop y Aprobación Semántica

El agente implementa un estricto control de seguridad que impide la ejecución desatendida de acciones masivas. El flujo consta de 4 etapas:

1. **Detección y Preparación:** En el nodo `prepare_action` (`backend/agente.py`), el agente compila el payload y lo guarda en `acciones_pendientes`.
2. **Notificación Reactiva:** FastAPI emite un evento SSE especial: `event: approval_required` conteniendo el identificador `action_id` y el resumen en JSON.

3. **Renderizado UI:** El frontend intercepta el evento y despliega el componente reactivo `TarjetaConfirmacion.vue` con botones de acción directa (`POST /chat/approve`).

4. **Evaluación Semántica de Lenguaje Natural:** Si el usuario decide responder directamente en la caja de texto (ej. *"dale"*, *"sí, procede"*, *.ªdelante con fe"*, *çancela eso"*), el módulo `backend/actions.py` utiliza el LLM para discernir la voluntad del operador:

```python
async def check_approval_semantic(text: str, pending_action: Optional[Dict[str, Any]] =
    None) -> Optional[bool]:
    quick = is_approval_text(text)
    if quick is not None:
        return quick
    if not pending_action:
        return None

    prompt = [{
        "role": "user",
        "content": (
            f"El sistema tiene una accion PREPARADA: {pending_action.get('preview')}\n"
            f"Mensaje del usuario: \"{text}\"\n"
            f"Determina si autoriza o cancela ESTA accion, o si esta dando una NUEVA
                orden. "
            f"Responde estrictamente: APROBADO, CANCELADO u OTRO."
        )
    }]
    res = await LLMService.chat_completion(messages=prompt, temperature=0.0)
    if "APROBADO" in res.upper():
        return True
    elif "CANCELADO" in res.upper():
        return False
    return None
```

Listing 9: Evaluador semántico de confirmación en backend/actions.py

9. Capa de Presentación y Streaming Token a Token

El cliente web (`frontend/`) es una Single Page Application (SPA) desarrollada en Vue 3 con Composition API, Pinia para el estado de autenticación y Vite como empaquetador.

9.1. Consumo Reactivo de Server-Sent Events

En `frontend/src/api.js`, el método `sendMessage` decodifica manualmente el stream HTTP utilizando la API estándar de `ReadableStream`:

```javascript
async sendMessage(conversacionId, mensaje, { onToken, onApprovalRequired, onDone, onError
    }) {
    const res = await fetch(`${API_BASE}/chat`, {
        method: 'POST',
        headers: getHeaders(),
        body: JSON.stringify({ conversacion_id: conversacionId, mensaje })
    });

    const reader = res.body.getReader();
    const decoder = new TextDecoder();
    let buffer = '';

    while (true) {
        const { value, done } = await reader.read();
        if (done) break;
        buffer += decoder.decode(value, { stream: true });

        const parts = buffer.split('\n\n');
        buffer = parts.pop(); // Conserva el fragmento incompleto

        for (const part of parts) {
            if (part.startsWith('event: token')) {
```

```
                const data = JSON.parse(part.split('data: ')[1]);
                onToken(data.content);
            } else if (part.startsWith('event: approval_required')) {
                const data = JSON.parse(part.split('data: ')[1]);
                onApprovalRequired(data);
            } else if (part.startsWith('event: done')) {
                onDone();
            }
        }
    }
}
```

Listing 10: Decodificación de chunks SSE en frontend/src/api.js

10. Conclusiones y Dictamen de Auditoría

El análisis exhaustivo del código fuente y los flujos operacionales permite emitir las siguientes conclusiones:

1. **Alineación Normativa Comprobada:** El agente operativiza con fidelidad los controles A.3.17 y A.3.18 de la norma ISO/IEC 27701:2025, transformando directivas abstractas en métricas calculables, plazos de vencimiento y requerimientos de firma de NDAs.

2. **Desacoplamiento Estándar (MCP + LangGraph):** La adopción de FastMCP separa limpiamente las funciones operativas del modelo de lenguaje, permitiendo auditorías de código independientes y sustitución de modelos sin alterar la lógica de negocio.

3. **Seguridad y Gobernanza de Datos:** El protocolo de aprobación humana (HITL) previene alucinaciones destructivas en comunicaciones masivas, y el registro de hashes SHA-256 en SQLite asegura el no repudio requerido por la Autoridad Nacional de Protección de Datos Personales (MINJUSDH).

4. **Resiliencia Operativa:** El diseño contempla fallbacks matemáticos y respuestas contextuales estructuradas, garantizando disponibilidad y servicio continuo incluso ante interrupciones de conectividad con proveedores de LLM externos.